

Geometry Utilities and uTimerScript

Geometry utilities and scripted timed posts.

Included MIT MOOS-IvP PDF sources:

- app_geometry.pdf
- app_utscrip.pdf

Geometry Utilities

June 2018

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139

1	Overview	1
2	General Geometric Object Properties	2
3	Points	3
3.1	String Representations for Points	3
4	Seglists	4
4.1	Standard String Representation for Seglists	4
4.2	The Lawnmower String Representation for Seglists	5
4.3	Seglists in the pMarineViewer Application	5
5	Polygons	6
5.1	Supported String Representation for Polygons	6
5.2	A Polygon String Representation using the Radial Format	7
5.3	A Polygon String Representation using the Ellipse Format	7
5.4	Optional Polygon Parameters	7
6	Seglrs	8
6.1	String Representations for Seglrs	8
6.2	Seglrs in the pMarineViewer Application	9
7	Vectors	9
7.1	String Representations for Vectors	9
7.2	Vectors in the pMarineViewer Application	10

1 Overview

This section discusses a few geometry data structures often used by the helm and the `pMarineViewer` application - convex polygons, lists of line segments, points, seglrs and vectors. These data structures are implemented by the classes `XYPolygon`, `XYSegList`, `XYPoint`, `XYSeglr` and `XYVector` respectively in the `lib_geometry` module distributed with the MOOS-IvP software bundle. The implementation of these class definitions is somewhat shielded from the helm user's perspective, but they are often involved in parameter settings of for behaviors. So the issue of how to specify a given geometric structure with a formatted string is discussed here.

Furthermore, the `pMarineViewer` application accepts these data structures for rendering by subscribing to three MOOS variables `VIEW_POLYGON`, `VIEW_SEGLIST`, `mVIEW_POINT`, and `VIEW_VECTOR`. These variables contain a string format representation of the structure, often with further visual hints on the color or size of the edges and vertices for rendering. These variables may originate from any MOOS application, but are also often posted by helm behaviors to provide visual clues

about what is going on in the vehicle. In the Alpha mission, for example, the waypoint behavior posted a seglist representing the set of waypoints for which it was configured, as well as posting a point indicating the next point on the behavior's list to traverse.

2 General Geometric Object Properties

Each of the four geometric objects, `XPolygon`, `XYSegList`, `XYPoint`, and `XYVector`, are a subclass of the general `XYObject` class, and share certain properties discussed here.

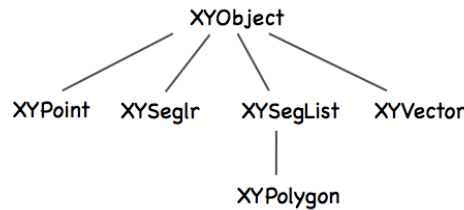


Figure 1: **Class hierarchy:** All geometric objects are subclasses of the `XYObject` class.

Common Properties

The following properties are defined at the `XYObject` level. All properties may be optionally left undefined by the user.

- *label*: A string that is often rendered in a GUI alongside the object rendering. In the `pMarineViewer` application, the label is also used as a unique identifier and successive receipt of geometric objects with the same label will result in the new one replacing” the older one. Thus the visual effect of ”moving” objects is rendered in this way.
- *msg*: A string typically regarded as an alternative to the label string when rendering an object. In the `pMarineViewer` application, if an object has a non-null *msg* field, this will be used for rendering the label instead of the *label* field.
- *type*: A string conveying some information on the type of the object from the source application. For example a point may be of type ”waypoint” or ”rendez-vous”, and so on.
- *time*: The time field is a double that may optionally be set to indicate when the point was generated, or how long it should exist before ”expiring”, or however an application may wish to interpret it.
- *source*: A string representing the source that generated the object. This may distinguish MOOS applications or helm behaviors for example.

Rendering Hint Properties

The following optional properties are defined at the `XYObject` level for providing hints to applications that may be using them for rendering.

- *active*: This is a Boolean that is regarded as `true` if left unspecified, and is used to indicate whether or not the object should be rendered. By setting this value to `false` for a given object, it would effectively be erased in the `pMarineViewer` application for example.
- *vertex_size*: This non-negative floating point value is a hint for how large to draw a vertex point for the given object. If left unspecified in the `pMarineViewer` application for example, the size of the vertex would be determined by the global setting for vertices set in the GUI.
- *edge_size*: This non-negative floating point value is a hint for how wide to draw an edge for the given object (if it has edges). If left unspecified in the `pMarineViewer` application for example, the width of the edge would be determined by the global setting for edges set in the GUI.
- *vertex_color*: This parameter specifies a hint or request to draw the vertices in this object in the given color specification. Colors may be specified by any defined color string or RGB string value as described in the Colors Appendix. In the `pMarineViewer` application, if this hint is not provided for received objects, the vertex color would be determined by the global setting for vertices set in the GUI. If the color is set to "invisible", this is effectively a request that the vertex not be rendered.
- *edge_color*: This parameter specifies a hint or request to draw the edges in this object in the given color specification. Colors may be specified by any defined color string or RGB string value as described in the Colors Appendix. In the `pMarineViewer` application, if this hint is not provided for received objects, the edge color would be determined by the global setting for edges set in the GUI. If the color is set to "invisible", this is effectively a request that the edge not be rendered.
- *label_color*: This parameter specifies a hint or request to draw the label for this object in the given color specification. Colors may be specified by any defined color string or RGB string value as described in the Colors Appendix. In the `pMarineViewer` application, if this hint is not provided for received objects, the label color would be determined by the global setting for labels set in the GUI. If the color is set to "invisible", this is effectively a request that label not be rendered.

3 Points

Points are implemented in the `XYPoint` class, and minimally represent a point in the x-y plane. These objects are used internally for applications and behaviors, and may also be involved in rendering in a GUI and therefore may have additional fields to support this as described in Section 2.

3.1 String Representations for Points

The only required information for a point specification is its position in the x-y plane. A third value may optionally be specified in the z-plane. If z is left unspecified, it will be set to zero. The standard string representation is a comma-separated list of param=value pairs like the following examples:

```

point    = x=60, y=-40
point    = x=60, y=-40, z=12
point    = z=19, y=5, x=23

```

Partly for backward compatibility, a very simplified string representation of a point is also supported:

```

point    = 60,-40
point    = 60, -40, 0

```

An example of string representation of point with all the optional parameters described in [Section 2](#) might look something like:

```

point    = x=60, y=-40, label=home, label_color=red, source=henry, type=waypoint,
          time=30, active=false, vertex_color=white, vertex_size=5, msg=bingo

```

Note that although a few different string formats are supported for *specifying* a point, only a single format is used when a `XYPoint` object is serialized into a string representation.

4 Seglists

Seglists are implemented in the `XYSegList` class and are comprised of an ordered set of vertices, implying line segments between each vertex. Seglist instances may be used for many things, but perhaps most often used to represent a set of vehicle waypoints. The geometry library has a number of basic operations defined for this class such as intersection testing with other objects, rotation, proximity testing and so on.

4.1 Standard String Representation for Seglists

The only requirement for a seglist specification is one or more vertex locations in the x, y plane. Values may optionally be specified in the z plane. If z is left unspecified it will default to zero. The standard string representation is a comma-separated list of param=value pairs like the following examples:

```

points   = pts={60,-40:60,-160:150,-160:180,-100:150,-40},label=foxtrot,type=top

```

By "standard" format we mean it is not only accepted as initialization input, but is also the format produced when an existing instance is serialized using the object's `string XYSegList::get_spec()` function. If z values are used, an example may look like the following which is the same as the above but associates $z = 2$ with each point:

```

points   = pts={60,-40,2:60,-160,2:150,-160,2:180,-100,2:150,-40,2},label=foxtrot

```

An example of a string representation with all the optional parameters described in Section 2 might look something like:

```
points = {pts=60,-40,2:60,-160,2:150,-160,2:180,-100,2:150,-40,2},label=foxtrot
          label_color=green, source=henry, type=return_path, time=30, active=false,
          vertex_color=white, vertex_size=5, edge_size=2, edge_color=red, msg=bingo
```

4.2 The Lawnmower String Representation for Seglists

Seglists may also be built using the `lawnmower` format. The following is an example:

```
points = format=lawnmower, label=foxtrot, x=0, y=40, height=60, width=180,
          lane_width=15, rows=north-south, startx=20, starty=-300, degs=45
```

The rotation of the pattern can optionally be specified in radians. For example, `degs=45` is equivalent to `rads=0.785398`. If, for some reason, both are specified, the seglist will be built using the `rads` parameter.

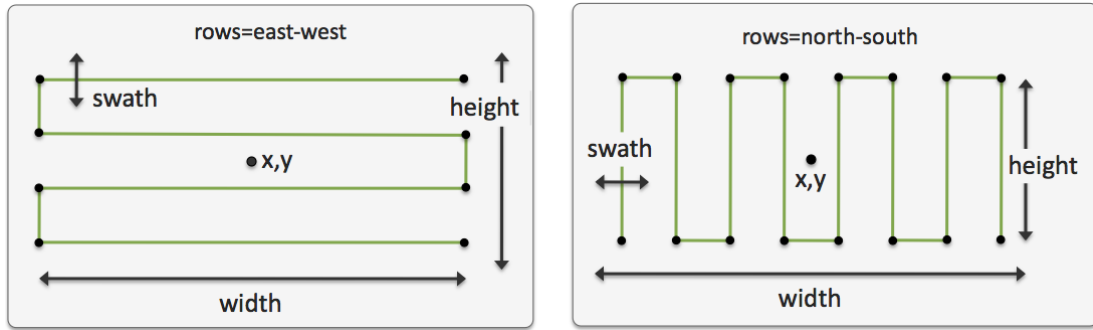


Figure 2: **Seglists built with the lawnmower format:** The pattern is specified by (a) the location of the center of the pattern, (b) the height and width of the pattern, (c) the lane width which determines the number of rows, (d) whether the pattern rows proceed north-south or east-west, and (e) an optional rotation of the pattern.

The `startx` and `starty` parameters name a point in the xy-plane to influence the starting position of the pattern. The seglist has four possible starting positions, and chosen starting position will be the position closest to the `(startx, starty)` position. The `startx` and `starty` parameters are optional and default to zero. Users sometimes may wish to use a vehicle's present x-y position for these parameters.

4.3 Seglists in the pMarineViewer Application

The `pMarineViewer` application registers for the MOOS variable `VIEW_SEGLIST`. The viewer maintains a list of seglists keyed on the label field of each incoming seglist. A seglist received with a thus far unique label will be added to the list of seglists rendered in the viewer. A seglist received with a non-unique label will replace the seglist with the same label in the memory of the viewer. This

has the effect of erasing the old seglist since each iteration of the viewer redraws everything, the background and all objects, from scratch several times per second.

The label is the only text rendered with the seglist in `pMarineViewer`. Since the label is also used as the key, if the user tries to "update" the label, or use the label to convey changing information, this may inadvertently result in accumulating multiple seglists in the viewer, each drawn over one another. Instead, the `VIEW_SEGLIST` may be posted with the message to be posted in the `msg=value` component. When this component is non-null, `pMarineViewer` renders it instead of the contents in the label component.

5 Polygons

Polygons are implemented in the `XYPolygon` class. This implementation accepts as a valid construction only specifications that build a convex polygon. Common operations used internally by behaviors and other applications, such as intersection tests, distance calculations etc, are greatly simplified and more efficient when dealing with convex polygons.

5.1 Supported String Representation for Polygons

The only requirement for a polygon specification is three or more vertex locations in the x, y plane constituting a convex polygon. Values may optionally be specified in the z plane. If z is left unspecified it will default to zero. The standard string representation is a comma-separated list of `param=value` pairs like the following examples:

```
points    = pts={60,-40:60,-160:150,-160:150,-40},label=foxtrot,type=one
```

By "standard" format we mean it is not only accepted as initialization input, but is also the format produced when an existing instance is serialized using the object's `string XYPolygon::get_spec()` function. If z values are used, an example may look like the following which is the same as the above but associates $z = 4$ with each point:

```
points    = pts={60,-40,4:60,-160,4:150,-160,4:150,-40,4},label=foxtrot,type=one
```

Polygons are defined by a set of vertices and the simplest way to specify the points is with a line comprised of a sequence of colon-separated pairs of comma-separated x-y points in local coordinates such as:

```
polygon   = 60,-40:60,-160:150,-160:180,-100:150,-40:label,foxtrot
```

If one of the pairs, such as the last one above, contains the keyword `label` on the left, then the value on the right, e.g., `foxtrot` as above, is the label associated with the polygon. An alternative notation for the same polygon is given by the following:

```
polygon   = label=foxtrot, pts={60,-40:60,-160:150,-160:180,-100:150,-40}
```

This is an comma-separated list of equals-separated pairs. The ordering of the comma-separated components is insignificant. The points describing the polygon are provided in braces to signify to the parser that everything in quotes is the right-hand side of the `pts=` component. Both formats are acceptable specifications of a polygon in a behavior for which there is a `polygon` parameter.

5.2 A Polygon String Representation using the Radial Format

Polygons may also be specified by their shape and the shape parameters. For example, a commonly used polygon is formed by points of an equal radial distance around a center point. The following is an example:

```
polygon = format=radial, label=foxtrot, x=0, y=40, radius=60, pts=6, snap=1
```

The `snap` component in the above example signifies that the vertices should be rounded to the nearest 1-meter value. The `x`, `y` parameters specify the middle of the polygon, and `radius` parameters specify the distance from the center for each vertex. The `pts` parameters specifies the number of vertices used, as shown in Figure 3.

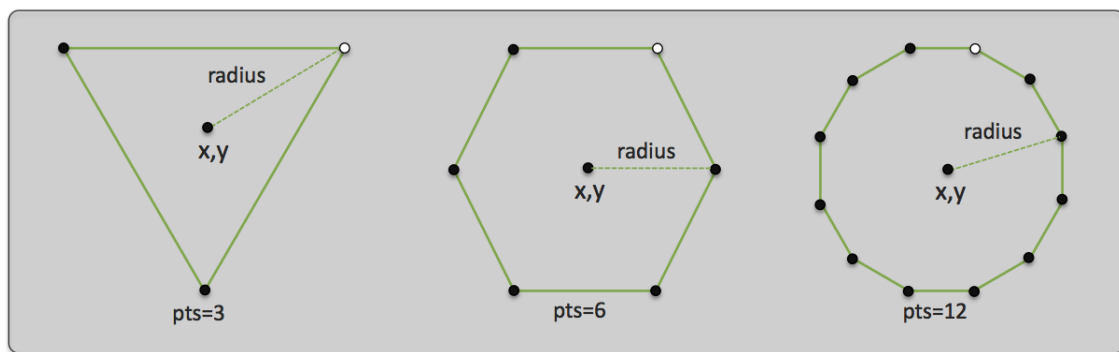


Figure 3: **Polygons built with the radial format:** Radial polygons are specified by (a) the location of their center, (b) the number of vertices, and (c) the radial distance from the center to each vertex. The lighter vertex in each polygon indicates the first vertex if traversing in sequence, proceeding clockwise.

5.3 A Polygon String Representation using the Ellipse Format

Polygons may also be built using the `ellipse` format. The following is an example:

```
polygon = label=golf, format=ellipse, x=0, y=40, degs=45, pts=14, snap=1,  
        major=100, minor=70
```

The `x`, `y` parameters specify the middle of the polygon, the `major` and `minor` parameters specify the radial distance of the major and minor axes. The `pts` parameters specifies the number of vertices used, as shown in Figure 4.

The rotation of the ellipse can optionally be specified in radians. For example, `degs=45` is equivalent to `rads=0.785398`. If, for some reason, both are specified, the polygon will be built using the `rads` parameter. When using the ellipse format, a minimum `pts=4` must be specified.

5.4 Optional Polygon Parameters

Polygons also may have several optional fields associated with them. The `label` field is string that is often rendered with a polygon in MOOS GUI applications such as the [pMarineViewer](#). The

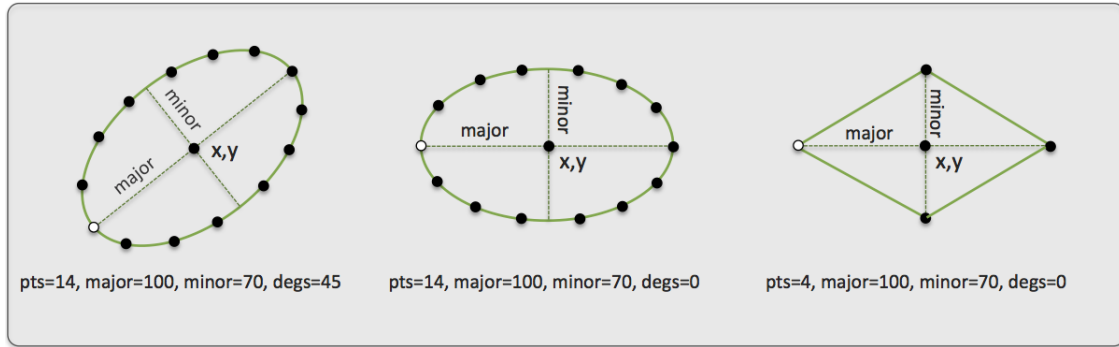


Figure 4: **Polygons built with the ellipse format:** Ellipse polygons are specified by (a) the location of their center, (b) the number of vertices, (c) the length of their major axis, (d) the length of their minor axis, and (e) the rotation of the ellipse. The lighter vertex in each polygon indicates the first vertex if traversing in sequence, proceeding clockwise.

`label_color` field represents a color preference for the label rendering. The `type` and `source` fields are additional string fields for further distinguishing a polygon in applications that handle them. The `active` field is a Boolean that is used in the `pMarineViewer` application to indicate whether the polygon should be rendered. The `time` field is a double that may optionally be set to indicate when the polygon was generated, or how long it should exist before "expiring", or however an application may wish to interpret it. The `vertex_color`, `edge_color`, and `vertex_size` fields represent further rendering preferences. The following are two equivalent further string representations:

```

polygon = format=radial, x=60, y=-40, radius=60, pts=8, snap=1, label=home,
          label_color=red, source=henry, type=survey, time=30, active=true,
          vertex_color=white, vertex_size=5, edge_size=2
polygon = format,radial:60,-40:radius,60:pts,8:snap,1:label,home:
          label_color,red:source,henry:type,survey:time,30:active,true:
          vertex_color,white:vertex_size,5:edge_size,2

```

The former is a more user-friendly format for specifying a polygon, perhaps found in a configuration file for example. The latter is the string representation passed around internally when `XYPolygon` objects are automatically converted to strings and back again in the code. This format is more likely to be found in log files or seen when scoping on variables with one of the MOOS scoping tools.

6 Seglrs

Seglrs are implemented in the `XYSeglr` class and are essentially comprised of list of line segments followed by a ray. They may be used in certain applications to indicate a ship maneuver. Minimally each seglr consists of at least one vertex in the x,y plane (but typically more, indicating a series of line segments), and the direction of the ray. They may also be configured with all relevant drawing hints discussed in Section 2.

6.1 String Representations for Seglrs

Seglr objects may be initialized from a string representation, and converted to a string representation from an existing object. The following is an example:

```
seglr = pts={5,5:30,10:90,-20},ray=45
```

The following is a string example using many of the general object fields and drawing hints available:

```
seglr = pts={5,5:30,10:90,-20},ray=45,          <-- defines the seglr
      ray_len=10,head_size=3,                    <-- seglr drawing hints
      label=alpha,edge_color=red,edge_size=2,     <-- general object
      vertex_size=3,vertex_color=green           <-- drawing hints
```

The parameters, `ray_len=10` and `"head_size=3"`, are drawing hints unique to the `seglr` object and refers to how big the arrow head should be rendered and how long the ray should be rendered. They are given in meters, and the default is `ray_len=10` and `head_size=3`.

6.2 Seglrs in the pMarineViewer Application

The `pMarineViewer` application registers for the MOOS variable `VIEW_SEGLR`. The viewer maintains a list of seglrs keyed on the label field of each incoming seglr. A seglr received with a thus far unique label will be added to the list of seglrs rendered in the viewer. A seglr received with a non-unique label will replace the seglr with the same label in the memory of the viewer. This has the effect of erasing the old seglr since each iteration of the viewer redraws everything, the background and all objects, from scratch several times per second.

The `VIEW_VECTOR` may be posted with the message label, to be posted just beyond the arrow head. This is done with the `msg` component of the posting. For example:

```
VIEW_SEGLR = pts={5,5:30,10:90,-20},ray=45,ray_len=10,head_size=3, \
            label=alpha,edge_color=red,edge_size=2,msg=alpha
```

7 Vectors

Vectors are implemented in the `XYVector` class and are essentially comprised of a location, direction and magnitude. They may be used in certain applications dealing with simulated or sensed forces, or simply used in a GUI application to render current fields or an instantaneous vehicle pose and trajectory. Minimally each vector consists of a location in the x, y plane and a direction and magnitude. They may also be configured with all relevant drawing hints discussed in Section 2.

7.1 String Representations for Vectors

Vector objects may be initialized from a string representation, and converted to a string representation from an existing object. The following is an example:

```
vector = x=5,y=10,ang=45,mag=20
```

Alternatively, instead of expressing the vector in terms of its direction and magnitude, it may also be given in terms of its magnitude in both the x and y direction. The following vector is nearly identical, modulo rounding errors, to the above configured vector:

```
vector = x=5,y=10,xdot=12.142,ydot=12.142
```

When a vector object is serialized to a string by invoking the object's native serialization function, the first of the two above formats will be used. The following is a string example using many of the general object fields and drawing hints available:

```
vector = x=5,y=10,ang=45,mag=20,label=pingu,source=simulator,type=wind,  
        vertex_size=2,vertex_color=red,edge_color=red,edge_size=2,head_size=12
```

The last parameter, "head_size=12", is a drawing hint unique to the vector object and refers to how big the arrow head should be rendered. If left unspecified, it may simply be rendered at whatever size the GUI application uses by default.

7.2 Vectors in the pMarineViewer Application

The `pMarineViewer` application registers for the MOOS variable `VIEW_VECTOR`. The viewer maintains a list of vectors keyed on the label field of each incoming vector. A vector received with a thus far unique label will be added to the list of vectors rendered in the viewer. A vector received with a non-unique label will replace the vector with the same label in the memory of the viewer. This has the effect of erasing the old vector since each iteration of the viewer redraws everything, the background and all objects, from scratch several times per second.

The label is the only text rendered with the vector in `pMarineViewer`. Since the label is also used as the key, if the user tries to "update" the label, or use the label to convey changing information, this may inadvertently result in accumulating multiple vectors in the viewer, each drawn over one another. Instead, the `VIEW_VECTOR` may be posted with the message to be posted in the `msg=value` component. When this component is non-null, `pMarineViewer` renders it instead of the contents in the label component.

uTimerScript: Scripting Events to the MOOSDB

June 2018

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
project-pavlab/appdocs/app_utscrip

1	Overview	1
2	Using uTimerScript	2
2.1	Configuring the Event List	2
2.2	Setting the Event Time or Range of Event Times	3
2.3	Resetting the Script	3
3	Script Flow Control	4
3.1	Pausing the Timer Script	4
3.2	Conditional Pausing of the Timer Script and Atomic Scripts	5
3.3	Fast-Forwarding the Timer Script	5
3.4	Quitting the Timer Script	5
4	Macro Usage in Event Postings	6
4.1	Built-In Macros Available	6
4.2	User Configured Macros with Random Variables	6
4.3	Support for Simple Arithmetic Expressions with Macros	7
5	Time Warps, Random Time Warps, and Restart Delays	7
5.1	Random Time Warping	7
5.2	Random Initial Start and Reset Delays	8
5.3	Status Messages Posted to the MOOSDB by uTimerScript	8
6	Terminal and AppCast Output	9
7	Configuration Parameters for uTimerScript	11
8	Publications and Subscriptions for uTimerScript	12
8.1	Variables Published by uTimerScript	12
8.2	Variables Subscribed for by uTimerScript	12
8.3	An Example MOOS Configuration Block	13
9	Examples	14
9.1	A Script for Generating 100 Random Numbers	14
9.2	A Script Used as Proxy for an On-Board GPS Unit	15
9.3	A Script as a Proxy for Simulating Random Wind Gusts	16

1 Overview

The `uTimerScript` application allows the user to script a set of pre-configured posts to a `MOOSDB`. In its most basic form, it may be used to initialize a set of variables to the `MOOSDB`, and immediately

terminate itself if a quit event is included. The following configuration block, if placed in the alpha example mission, would mimic the posts to the `MOOSDB` behind the `DEPLOY` button, simply disabling manual control, deploying the vehicle and quitting the script.

Listing 1.1: A Simple Timer Script.

```
1 ProcessConfig = uTimerScript
2 {
3     event = var=MOOS_MANUAL_OVERRIDE, val=false
4     event = var=DEPLOY, val=true
5     event = quit
6 }
```

Additionally, `uTimerScript` may be used with the following advanced functions:

- Each entry in the script may be scheduled to occur after a specified amount of elapsed time.
- Event timestamps may be given as an exact point in time relative to the start of the script, or a range in times with the exact time determined randomly at run-time.
- The execution of the script may be paused, or fast-forwarded a given amount of time, or forwarded to the next event on the script by writing to a MOOS variable.
- The script may be conditionally paused based on user defined logic conditions over one or more MOOS variables.
- The variable value of an event may also contain information generated randomly.
- The script may be reset or repeated any given number of times.
- The script may use its own time warp, which can be made to vary randomly between script executions.

In short, `uTimerScript` may be used to effectively simulate the output of other MOOS applications when those applications are not available. A few examples are provided, including a simulated GPS unit and a crude simulation of wind gusts.

2 Using uTimerScript

Configuring a script minimally involves the specification of one or more events, with an event comprising of a MOOS variable and value to be posted and an optional time at which it is to be posted. Scripts may also be reset on a set policy, or from a trigger by an external process.

2.1 Configuring the Event List

The event list or script is configured by declaring a set of event entries with the following format:

```
event = var=<MOOSVar>, val=<value>, [time=<time-of-event>]
```

The keywords `event`, `var`, `val`, and `time` are not case sensitive, but the values `<moos-variable>` and `<var-value>` are case sensitive. The `<var-value>` type is posted either as a string or double based on the following heuristic: if the `<var-value>` has a numerical value it is posted as a double, and

otherwise posted as a string. If one wants to post a string with a numerical value, putting quotes around the number suffices to have it posted as a string. Thus `val=99` posts a double, but `var="99"` posts a string. If a string is to be posted that contains a comma such as "apples, pears", one must put the quotes around the string to ensure the comma is interpreted as part of `<var-value>`. The value field may also contain one or more macros expanded at the time of posting, as described in Section 4.

2.2 Setting the Event Time or Range of Event Times

The value of `<time-of-event>` is given in seconds and must be a numerical value greater or equal to zero. The time represents the amount of elapsed time since the `uTimerScript` was first launched and un-paused. The list of events provided in the configuration block need not be in order - they will be ordered by the `uTimerScript` utility. The `<time-of-event>` may also be specified by a interval of time, e.g., `time=0:100`, such that the event may occur at some point in the range with uniform probability. The only restrictions are that the lower end of the interval is greater or equal to zero, and less than or equal to the higher end of the interval. By default the timestamps are calculated once from their specified interval, at the the outset of `uTimerScript`. The script may alternatively be configured to recalculate the timestamps from their interval each time the script is reset, by setting the `shuffle` parameter to true. This parameter, and resetting in general, are described in the next Section 2.3.

2.3 Resetting the Script

The timer script may be reset to its initial state, resetting the stored elapsed-time to zero and marking all events in the script as pending. This may occur either by cueing from an event outside `uTimerScript`, or automatically from within `uTimerScript`. Outside-cued resets can be triggered by posting `UTS_RESET` with the value `"reset"`, or `"true"`. The `reset_var` parameter names a MOOS variable that may be used as an alternative to `UTS_RESET`. It has the format:

```
reset_var = <moos-variable> // Default is UTS_RESET
```

The script may be also be configured to auto-reset after a certain amount of time, or immediately after all events are posted, using the `reset_time` parameter. It has the format:

```
reset_time = <time-or-condition> // Default is "none"
```

The `<time-or-condition>` may be set to `"all-posted"` which will reset after the last event is posted. If set to a numerical value greater than zero, it will reset after that amount of elapsed time, regardless of whether or not there are pending un-posted events. If set to `"none"`, the default, then no automatic resetting is performed. Regardless of the `reset_time` setting, prompted resets via the `UTS_RESET` variable may take place when cued.

The script may be configured to accept a hard limit on the number of times it may be reset. This is configured using the `reset_max` parameter and has the following format:

```
reset_max = <amount> // Default is "nolimit"
```

The `<amount>` specified may be any number greater or equal to zero, where the latter, in effect, indicates that no resets are permitted. If unlimited resets are desired (the default), the case

insensitive argument "unlimited" or "any" may be used.

The script may be configured to recalculate all event timestamps specified with a range of values whenever the script is reset. This is done with the following parameter:

```
shuffle = false // Default is "true"
```

The script may be configured to reset or restart each time it transitions from a situation where its conditions are not met to a situation where its conditions are met, or in other words, when the script is "awoken". The use of logic conditions is described in more detail in Section 3.1. This is done with the following parameter:

```
upon_awake = restart // Default is "n/a", no action
```

Note that this does not apply when the script transitions from being paused to un-paused as described in Section 3.1. See the example in Section 9.2 for a case where the `upon_awake` feature is handy.

3 Script Flow Control

The script flow may be affected in a number of ways in addition to the simple passage of time. It may be (a) paused by explicitly pausing it, (b) implicitly paused by conditioning the flow on one or more logic conditions, (c) fast-forwarded directly to the next scheduled event, or fast-forwarded some number of seconds. Each method is described in this section.

3.1 Pausing the Timer Script

The script can be paused at any time and set to be paused initially at start time. The `paused` parameter affects whether the timer script is actively unfolding at the outset of launching `uTimerScript`. It has the following format:

```
paused = <Boolean>
```

The keyword `paused` and the string representing the Boolean are not case sensitive. The Boolean simply must be either "true" or "false". By setting `paused` to true, the elapsed time calculated by `uTimerScript` is paused and no variable-value pairs will be posted. When un-paused the elapsed time begins to accumulate and the script begins or resumes unfolding. The default value of `paused` is false.

The script may also be paused through the MOOS variable `UTS_PAUSE` which may be posted by some other MOOS application. The values recognized are "true", "false", or "toggle", all case insensitive. The name of this variable may be substituted for a different one with the `pause_var` parameter in the `uTimerScript` configuration block. It has the format:

```
pause_var = <MOOSVar> // Default is UTS_PAUSE
```

If multiple scripts are being used (with multiple instances of `uTimerScript` connected to the `MOOSDB`), setting the `pause_var` to a unique variable may be needed to avoid unintentionally pausing or un-pausing multiple scripts with single write to `UTS_PAUSE`.

3.2 Conditional Pausing of the Timer Script and Atomic Scripts

The script may also be configured to condition the "paused-state" to depend on one or more logic conditions. If conditions are specified in the configuration block, the script must be both un-paused as described above in Section 3.1, and all specified logic conditions must be met in order for the script to begin or resume proceeding. The logic conditions are configured as follows:

```
condition = <logic-expression>
```

The <logic-expression> syntax is described in Logic Appendix, and may involve the simple comparison of MOOS variables to specified literal values, or the comparison of MOOS variables to one another. See the script configuration in Section 9.2 for one example usage of logic expressions.

An *atomic* script is one that does not check conditions once it has posted its first event, and prior to posting its last event. Once a script has started, it is treated as unpausable with respect to the logic conditions. This is configured with:

```
script_atomic = <Boolean>
```

It can however be paused and unpaused via the pause variable, e.g., `UTS_PAUSE`, as described in Section 3.1. If the logic conditions suddenly fail in an atomic script midway, the check is simply postponed until after the script completes and is perhaps reset. If the conditions in the meanwhile revert to being satisfied, then no interruption should be observable.

3.3 Fast-Forwarding the Timer Script

The timer script, when un-paused, moves forward in time with events executed as their event times arrive. However, the script may be moved forward by writing to the MOOS variable `UTS_FORWARD`. If the value received is zero (or negative), the script will be forwarded directly to the point in time at which the next scheduled event occurs. If the value received is positive, the elapsed time is forwarded by the given amount. Alternatives to the MOOS variable `UTS_FORWARD` may be configured with the parameter:

```
forward_var = <MOOSVar> // Default is UTS_FORWARD
```

If multiple scripts are being used (with multiple instances of `uTimerScript` connected to the `MOOSDB`), setting the `forward_var` to a unique variable may be needed to avoid unintentionally fast forwarding multiple scripts with single write to `UTS_FORWARD`.

3.4 Quitting the Timer Script

The timer script may be configured with a special event, the *quit* event, resulting in disconnection with the `MOOSDB` and a process exit. This is done with the configuration:

```
event = quit [time=<time-of-event>]
```

Before quitting, a final posting to the `MOOSDB` is made with the variable `EXITED_NORMALLY`. The value is `"uTimerScript"`, or its alias if an alias was used. This indicates to any other watchdog process, such as `uProcessWatch`, that the exiting of this script is not a reason for concern. When `uTimerScript` receives its own posting in the next incoming mail, it assumes all pending posts have been made

and will then quit.

4 Macro Usage in Event Postings

Macros may be used to add a dynamic component to the value field of an event posting. This substantially expands the expressive power and possible uses of the `uTimerScript` utility. Recall that the components of an event are defined by:

```
event = var=<MOOSVar>, val=<var-value>, time=<time-of-event>
```

The `<var-value>` component may contain a macro of the form `$(MACRO)`, where the macro is either one of a few built-in macros available, or a user-defined macro with the ability to represent random variables. Macros may also be combined in simple arithmetic expressions to provide further expressive power. In each case, the macro is expanded at the time of the event posting, typically with different values on each successive posting.

4.1 Built-In Macros Available

There are five built-in macros available:

- `$(DBTIME)`: The estimated time since the `MOOSDB` started, similar to the value in the MOOS variable `DE_UPTIME` published by the `MOOSDB`.

An example usage:

```
event = var=DEPLOY_RECEIVED, val=$(DBTIME), time=10:20
```

- `$(UTCTIME)`: The UTC time, in seconds, at the time of the posting.
- `$(UTC)`: The UTC time, in seconds, at the time of the posting.
- `$(COUNT)`: Expands to the integer total of all posts thus far in the current execution of the script, and is reset to zero when the script resets.
- `$(TCOUNT)`: Expands to the integer total of all posts thus far since the application began, i.e., it is a running total that is not reset when the script is reset.
- `$(IDX)`: Expands to the integer value representing an event's count or index into the sequence of events. However, it will always post as a string and will be padded with zeros to the left, e.g., "000", "001", ..., and so on.

If the value of a MOOS posting is solely a macro, the variable type of the posting is a double, not a string. For example `val=$(DBTIME)` will post a type double, whereas `val="time:$(DBTIME)"` will post a type string.

4.2 User Configured Macros with Random Variables

Further macros are available for use in the `<var-value>` component of an event, defined and configured by the user, and based on the idea of a random variable. In short, the macro may expand to a

numerical value chosen within a user specified range, and recalculated according to a user-specified policy. The general format is:

```
rand_var = varname=<variable>, min=<value>, max=<value>, key=<key_name>
```

The <variable> component defines the macro name. The <low_value> and <high_value> components define the range from which the random value will be chosen uniformly. The <key_name> determines when the random value is reset. It must be set to one of the following three values: "at_start", "at_reset", and "at_post". Random variables with the key name "at_start" are assigned a random value only at the start of the `uTimerScript` application. Those with the "at_reset" key name also have their values re-assigned whenever the script is reset. Those with the "at_post" key name also have their values re-assigned after any event is posted.

4.3 Support for Simple Arithmetic Expressions with Macros

Macros that expand to numerical values may be combined in simple arithmetic expressions with other macros or scalar values. The general form is:

```
{<value> <operator> <value>}
```

The <value> components may be either a scalar or a macro, and the <operator> component may be one of '+', '-', '*', '/'. Nesting is also supported. Below are some examples:

```
{${FOOBAR} * 0.5}  
{-2-${FOOBAR}}  
{${APPLES} + ${PEARS}}  
{35 / {${FOOBAR}-2}}  
{${DBTIME} - {35 / {${UTCTIME}+2}}}
```

If a macro should happen to expand to a string rather than a double (numerical) value, the string evaluates to zero for the sake of the remaining evaluations.

5 Time Warps, Random Time Warps, and Restart Delays

A time warp and initial start delay may be optionally configured into the script to change the event schedule without having to edit all the time entries for each event. They may also be configured to take on a new random value at the outset of each script execution to allow for simulation of events in nature or devices having a random component.

5.1 Random Time Warping

The time warp is a numerical value in the range $(0, \infty]$, with a default value of 1.0. Lower values indicate that time is moving more slowly. As the script unfolds, a counter indicating "elapsed_time" increases in value as long as the script is not paused. The "elapsed_time" is multiplied by the time warp value. The time warp may be specified as a single value or a range of values as below:

```
time_warp = <value>
time_warp = <low-value>:<high-value>
```

When a range of values is specified, the time warp value is calculated at the outset, and re-calculated whenever the script is reset. See the example in Section 9.3 for a use of random time warping to simulate random wind gusts.

5.2 Random Initial Start and Reset Delays

A start delay may be provided with the `delay_start` parameter, given in seconds in the range $[0, \infty]$, with a default value of 0. The effect of having a non-zero delay of n seconds is to have `elapsed_time=n` at the outset of the script, on the first time through the script only. Thus a delay of n seconds combined with a time warp of 0.5 would result in observed delay of $2 * n$ seconds. The start delay may be specified as a single value or a range of values as below:

```
delay_start = <value>
delay_start = <low-value>:<high-value>
```

To specify a delay applied at the beginning if the script *after a reset*, use the `delay_reset` parameter instead.

```
delay_reset = <value>
delay_reset = <low-value>:<high-value>
```

When a range of values is specified, the start or reset delay value is calculated at the outset, and re-calculated whenever the script is reset. See the example in Section 9.2 for a use of random start delays to simulate the delay in acquiring satellite fixes in a GPS unit on an UUV coming to the surface.

5.3 Status Messages Posted to the MOOSDB by uTimerScript

The `uTimerScript` periodically publishes a string to the MOOS variable `UTS_STATUS` indicating the status of the script. This variable will be published on each iteration if one of the following conditions is met: (a) two seconds has passed since the previous status message posted, or (b) an event has been posted, or (c) the paused state has changed, or (d) the script has been reset, or (e) the state of script logic conditions has changed. A posting may look something like:

```
UTS_STATUS = "name=RND_TEST, elapsed_time=2.00, posted=1, pending=5, paused=false,
conditions_ok=true, time_warp=3, start_delay=0, shuffle=false,
upon_awake=restart, resets=2/5"
```

In this case, the script has posted one of six events (`posted=1`, `pending=5`). It is actively unfolding, since `paused=false` (Section 3.1) and `conditions_ok=true` (Section 3.2). It has been reset twice out of a maximum of five allowed resets (`resets=2/5`, Section 2.3). Time warping is being deployed `time_warp=3` (Section 5), there is no start delay in use `start_delay=0` (Section 5.2). The shuffle feature

is turned off `shuffle=false` (Section 2.3). The script is not configured to reset upon re-entering the un-paused state, `awake_reset=false` (Section 2.3).

When multiple scripts are running in the same MOOS community, one may want to take measures to discern between the status messages generated across scripts. One way to do this is to use a unique MOOS variable other than `UTS_STATUS` for each script. The variable used for publishing the status may be configured using the `status_var` parameter. It has the following format:

```
status_var = <MOOSVar> // Default is UTS_STATUS
```

Alternatively, a unique name may be given to each to each script. All status messages from all scripts would still be contained in postings to `UTS_STATUS`, but the different script output could be discerned by the name field of the status string. The script name is set with the following format.

```
script_name = <string> // Default is "unnamed"
```

6 Terminal and AppCast Output

The script configuration and progress of script execution may also be monitored from an open console window where `uTimerScript` is launched, or through an appcast viewer. Example output is shown below in Listing 2. On line 2, the name of the local community or vehicle name is listed on the left. On the right, "0/0(450)" indicates there are no configuration or run warnings, and the current iteration of `uFldTimerScript` is 450.

Lines 4-16: Script Configuration

Lines 4-11 show the script configuration. Line 5 shows the number of elements in the script and in parentheses the last element to have been posted. Line 6 shows the number of times the script has restarted. Line 7 shows the present time warp and the range of time warps possible on each script restart in brackets (Section 5.1). Lines 8-9 show the delay applied at the start and after a script reset (Section 5.2). Line 10 indicates the script is presently not paused (Section 3.1). Line 11 indicates the script presently meets any prevailing logic conditions (Section 3.2). Lines 13-16 show that there are two random variables defined for this script that may be used in event definitions. They are both uniform random variables. The first varies over possible directions, and the second over possible speed magnitudes. Section 4.2.

```

1 =====
2 uTimerScript charlie
3 =====
4 Current Script Information:
5     Elements: 10(8)
6     Reinit: 2
7     Time Warp: 1.09 [0.2,2]
8     Delay Start: 0
9     Delay Reset: 23.66 [10,60]
10    Paused: false
11 ConditionsOK: true
12
13 RandomVar  Type      Min  Max  Parameters
14 -----
15 ANG        uniform  0    359
16 MAG        uniform  1.5  3.5
17
18 P/Tot  P/Loc  T/Total  T/Local  Variable/V
19 -----
20 19      9    196.77   72.15   DRIFT_VECT
21 20      0    219.42   24.08   DRIFT_VECT
22 21      1    220.93   25.71   DRIFT_VECT
23 22      2    222.95   27.91   DRIFT_VECT
24 23      3    224.96   30.09   DRIFT_VECT
25 24      4    226.46   31.73   DRIFT_VECT
26 25      5    228.47   33.91   DRIFT_VECT
27 26      6    230.49   36.10   DRIFT_VECT
28
29 =====
30 Most Recent Events (3):
31 =====
32 [192.78]: Script Re-Start. Warp=0.48922, Del
33 [44.80]: Script Re-Start. Warp=1.61692, Del
34 [0.51]: Script Start. Warp=1, DelayStart=0.

```

Listing 6.2: Example *uTimerScript* console and *appcast* output.

Lines 18-27: Recent Script Postings

Lines 18-27 show recent postings to the **MOOSDB** by the script. The first column shows the total postings so far for the script. The second column shows the index within the script. In the above example, there are ten elements in the script. The most recent posting on line 27, shows the script has been reset twice and the most recent posting is of the seventh element of the script (index 6). The third column shows the total time since script started, and the fourth column shows the time since the script was re-started. Note the time delay between lines 20 and 21, due to the **delay_reset** shown on line 9. The last column shows the actual variable value pair posted.

Lines 29-34: Recent Events

Lines 29-34 show recent events (other than event postings). In this case it shows the script has been started, and re-started twice. Notice the delay reset on line 32 is different than that on line 9. The delay reset time of 23.66 seconds shown on line 9 is the delay reset to be applied on the *next* reset.

7 Configuration Parameters for uTimerScript

The following parameters are defined for `uTimerScript`. A more detailed description is provided in other parts of this section. Parameters having default values are indicated.

Listing 7.3: Configuration Parameters for uTimerScript.

<code>block_on:</code>	A comma-separated list of MOOS apps on which the script will block until seen in the list of <code>DB_CLIENTS</code> .
<code>condition:</code>	A logic condition that must be met for the script to be un-paused. Section 3.2.
<code>delay_reset:</code>	Number of seconds added to each event time, on each script reset. Legal values: any non-negative numerical value, or range of values separated by a colon. The default is zero. Section 5.2.
<code>delay_start:</code>	Number of seconds, or range of seconds, added to each event time, on first pass only. Legal values: any non-negative numerical value, or range of values separated by a colon. The default is zero. Section 5.2.
<code>event:</code>	A description of a single event in the timer script. Section 2.1.
<code>forward_var:</code>	A MOOS variable for taking cues to forward time. The default is <code>UTS_FORWARD</code>). Section 3.3.
<code>paused:</code>	A Boolean indicating whether the script is paused upon launch. Legal values: true, false. The default is false. Section 3.1.
<code>pause_var:</code>	A MOOS variable for receiving pause state cues (<code>UTS_PAUSE</code>). Section 3.1.
<code>rand_var:</code>	A declaration of a random variable macro to be expanded in event values. Section 4.2.
<code>reset_max:</code>	The maximum amount of resets allowed. Legal values: any non-negative integer, or the string "nolimit". The default is "nolimit". NOTE: If re-setting is desired, the <code>reset_time</code> parameter must also be changed from its default value of "none". Section 2.3.
<code>reset_time:</code>	The time or condition when the script is reset Legal values: Any non-negative number, the string "none", or "all-posted", or equivalently, "end". The default is "none". Section 2.3.
<code>reset_var:</code>	A MOOS variable for receiving reset cues. The default is <code>UTS_RESET</code> .
<code>script_atomic:</code>	When true, a started script will complete if conditions suddenly fail. Legal values: true, false. The default is false.
<code>script_name:</code>	Unique (hopefully) name given to this script. The default is "unnamed".
<code>shuffle:</code>	If true, timestamps are recalculated on each reset of the script. Legal values: true, false. The default is true. Section 2.3.
<code>status_var:</code>	A MOOS variable for posting status summary. The default is <code>UTS_STATUS</code> . Section 5.3
<code>time_warp:</code>	Rate at which time is accelerated in executing the script. Legal values: any non-negative number. The default is zero. Section 5.

- `time_zero`: The base time upon which script event times are based. The default is "script_start", the time at which `uTimerScript` is launched. The one alternative is "db_start", the time at which the MOOSDB was launched. The default is "script_start".
- `upon_awake`: Reset or re-start the script upon conditions being met after failure ("n/a"). Section 2.3.
- `verbose`: If true, progress output is generated to the console (true).

8 Publications and Subscriptions for uTimerScript

The interface for `uTimerScript`, in terms of publications and subscriptions, is described below. This same information may also be obtained from the terminal with:

```
$ uTimerScript --interface or -i
```

8.1 Variables Published by uTimerScript

The primary output of `uTimerScript` to the MOOSDB is the set of configured events, but one other variable is published on each iteration, and another upon purposeful exit with the `event=quit` event configuration.

- `APPCAST`: Contains an appcast report identical to the terminal output. Appcasts are posted only after an appcast request is received from an appcast viewing utility. Section 6.
- `EXITED_NORMALLY`: A posting made when the script contains and executes a `event=quit` event, to let other applications know that the disconnection of `uTimerScript` is not a concern for alarm.
- `UTS_STATUS`: A status string of script progress. Section 5.3.

8.2 Variables Subscribed for by uTimerScript

The `uTimerScript` application will subscribe for the following four MOOS variables to provide optional control over the flow of the script by the user or other MOOS processes:

- `APPCAST_REQ`: A request to generate and post a new appcast report, with reporting criteria, and expiration.
- `EXITED_NORMALLY`: When `uTimerScript` receives its own posting, it is assumed that all outgoing posts needed to be made before quitting have been received by the MOOSDB. Upon this receipt `uTimerScript` will quit. See Section 3.4.
- `UTS_NEXT`: When received with the value "next", the script will fast-forward in time to the next event. See Section 3.3.
- `UTS_RESET`: When received with the value of either "true" or "reset", the timer script will be reset. See Section 2.3.
- `UTS_FORWARD`: When received with a numerical value greater than zero, the script will fast-forward by the indicated time. See Section 3.3.

- **UTS_PAUSE**: When received with the value of "true", "false", "toggle", the script will change its pause state correspondingly. See Section 3.1.

In addition to the above MOOS variables, **uTimerScript** will subscribe for any variables involved in logic conditions, described in Section 3.2.

8.3 An Example MOOS Configuration Block

To see an example MOOS configuration block, enter the following from the command-line:

```
$ uTimerScript --example
```

This will show the output shown in Listing 4 below.

Listing 8.4: Example configuration of the uTimerScript application.

```

1  =====
2  uTimerScript Example MOOS Configuration
3  =====
4  Blue lines:      Default configuration
5
6  ProcessConfig = uTimerScript
7  {
8      AppTick      = 4
9      CommsTick    = 4
10
11     // Logic condition that must be met for script to be unpaused
12     condition      = WIND_GUSTS = true
13     // Seconds added to each event time, on each script pass
14     delay_reset    = 0
15     // Seconds added to each event time, on first pass only
16     delay_start    = 0
17     // Event(s) are the key components of the script
18     event           = var=SBR_RANGE_REQUEST, val="name=archie", time=25:35
19     // A MOOS variable for taking cues to forward time
20     forward_var     = UTS_FORWARD // or other MOOS variable
21     // If true script is paused upon launch
22     paused          = false // or {true}
23     // A MOOS variable for receiving pause state cues
24     pause_var       = UTS_PAUSE // or other MOOS variable
25     // Declaration of random var macro expanded in event values
26     randvar         = varname=ANG, min=0, max=359, key=at_reset
27     // Maximum number of resets allowed
28     reset_max       = nolimit // or in range [0,inf)
29     // A point when the script is reset
30     reset_time      = none // or {all-posted} or range (0,inf)
31     // A MOOS variable for receiving reset cues
32     reset_var       = UTS_RESET // or other MOOS variable
33     // If true script will complete if conditions suddenly fail
34     script_atomic   = false // or {true}
35     // A hopefully unique name given to the script
36     script_name     = unnamed
37     // If true timestamps are recalculated on each script reset

```



```

38  shuffle      = true
39  // If true progress is generated to the console
40  verbose      = true    // or {false}
41  // Reset or restart script upon conditions being met after failure
42  upon_awake   = n/a     // or {reset,resstart}
43  // A MOOS variable for posting the status summary
44  status_var   = UTS_STATUS // or other MOOS variable
45  // Rate at which time is accelerated in executing the script
46  time_warp    = 1
47  }

```

9 Examples

The examples in this section demonstrate the constructs thus far described for the `uTimerScript` application. In each case, the use of the script obviated the need for developing and maintaining a separate dedicated MOOS application.

9.1 A Script for Generating 100 Random Numbers

The below script will generate 100 postings with random numbers preceded by an initial "start" posting, and followed by an "end" posting.

Listing 9.5: A uTimerScript configuration for generating 100 random numbers.

```

//-----
ProcessConfig = uTimerScript
{
  rand_var    = varname=RND_VAL, min=0, max=50, key=at_post

  event = var=REPORT, val="start", time=0
  event = var=REPORT, val="accidents=${RND_VAL}, unique_id=${TCOUNT}", time=0, amt=100
  event = var=REPORT, val="end", time=0
}

```

The above may result in postings like those below (take from analog file):

```

8.817    REPORT      uTimerScript    start
8.817    REPORT      uTimerScript    accidents=35.234,unique_id=1
8.817    REPORT      uTimerScript    accidents=32.01,unique_id=2
8.818    REPORT      uTimerScript    accidents=26.982,unique_id=3
...
8.856    REPORT      uTimerScript    accidents=32.3,unique_id=95
8.857    REPORT      uTimerScript    accidents=13.324,unique_id=96
8.857    REPORT      uTimerScript    accidents=27.624,unique_id=97
8.858    REPORT      uTimerScript    accidents=28.901,unique_id=98
8.858    REPORT      uTimerScript    accidents=18.9,unique_id=99
8.859    REPORT      uTimerScript    accidents=35.72,unique_id=100
8.859    REPORT      uTimerScript    end

```

To make the numerical values integers, use the `snap=1` option (available in the next release after 19.8). Using `snap=0.1` will round to the nearest tenth and so on. For example:

```
rand_var = varname=RND_VAL, min=0, max=50, key=at_post, snap=1
```

9.2 A Script Used as Proxy for an On-Board GPS Unit

Typical operation of an underwater vehicle includes the periodic surfacing to obtain a GPS fix to correct navigation error accumulated while under water. A GPS unit that has been out of satellite communication for some period normally takes some time to re-acquire enough satellites to resume providing position information. From the perspective of the helm and configuring an autonomy mission, it is typical to remain at the surface only long enough to obtain the GPS fix, and then resume other aspects of the mission at-depth.

Consider a situation as shown in Figure 1, where the autonomy system is running in the payload on a payload computer, receiving not only updated navigation positions (in the form of `NAV_DEPTH`, `NAV_X`, and `NAV_Y`), but also a "heartbeat" signal each time a new GPS position has been received (`GPS_RECEIVED`). This heartbeat signal may be enough to indicate to the helm and mission configuration that the objective of the surface excursion has been achieved.

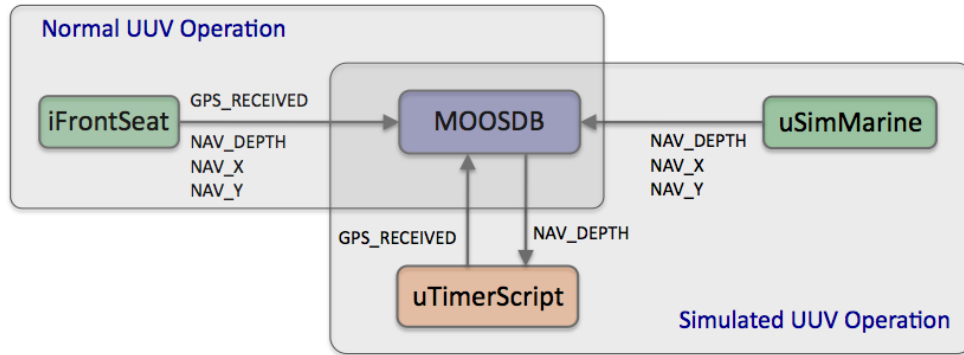


Figure 1: **Simulating a GPS Acknowledgment:** In a physical operation of the vehicle, the navigation solution and a `GPS.UPDATE.RECEIVED` heartbeat are received from the main vehicle (front-seat) computer via a MOOS module acting as an interface to the front-seat computer. In simulation, the navigation solution is provided by the simulator without any `GPS.UPDATE.RECEIVED` heartbeat. This element of simulation may be provided with `uTimerScript` configured to post the heartbeat, conditioned on the `NAV_DEPTH` information and a user-specified start delay to simulate GPS acquisition delay.

In simulation, however, the simulator only produces a steady stream of navigation updates with no regard to a simulated GPS unit. At this point there are three choices: (a) modify the simulator to fake GPS heartbeats and satellite delay, (b) write a separate simple MOOS application to do the same simulation. The drawback of the former is that one may not want to branch a new version of the simulator, or even introduce this new complexity to the simulator. The drawback of the latter is that, if one wants to propagate this functionality to other users, this requires distribution and version control of a new MOOS application.

A third and perhaps preferable option (c) is to write a short script for `uTimerScript` simulating the desired GPS characteristics. This achieves the objectives without modifying or introducing new source code. The below script in Listing 6 gets the job done.

Listing 9.6: A `uTimerScript` configuration for simulating aspects of a GPS unit.

```
1  //-----
2  // uTimerScript configuration block
3
4  ProcessConfig = uTimerScript
5  {
6      AppTick    = 4
7      CommsTick  = 4
8
9      paused     = false
10     reset_max  = unlimited
11     reset_time = end
12     condition  = NAV_DEPTH < 0.2
13     upon_awake = restart
14     delay_start = 20:120
15     script_name = GPS_SCRIPT
16
17     event = var=GPS_UPDATE_RECEIVED, val="RCVD_${COUNT}", time=0:1
18 }
```

This script posts a `GPS.UPDATE.RECEIVED` heartbeat message roughly once every second, based on the event time "time=0:1" on line 17. The value of this message will be unique on each posting due to the `_${COUNT}` macro in the value component. See Section 4.1 for more on macros. The script is configured to restart each time it awakes (line 13), defined by meeting the condition of (`NAV_DEPTH < 0.2`) which is a proxy for the vehicle being at the surface. The `delay_start` simulates the time needed for the GPS unit to reacquire satellite signals and is configured to be somewhere in the range of 20 to 120 seconds (line 14). Once the script gets past the start delay, the script is a single event (line 17) that repeats indefinitely since `reset_max` is set to `unlimited` and `reset_time` is set to `end` in lines 10 and 11. This script is used in the IvP Helm example simulation mission labeled "s4.delta" illustrating the `PeriodicSurface` helm behavior.

9.3 A Script as a Proxy for Simulating Random Wind Gusts

Simulating wind gusts, or in general, somewhat random external periodic drift effects on a vehicle, are useful for testing the robustness of certain autonomy algorithms. Often they don't need to be grounded in very realistic models of the environment to be useful, and here we show how a script can be used simulate such drift effects in conjunction with the `uSimMarine` application.

The `uSimMarine` application is a simple simulator that produces a stream of navigation information, `NAV_X`, `NAV_Y`, `NAV_SPEED`, `NAV_DEPTH`, and `NAV_HEADING` (Figure 2), based on the vehicle's last known position and trajectory, and currently observed values for actuator variables. The simulator also stores local state variables reflecting the current external drift in the x-y plane, by default zero. An external drift may be specified in terms of a drift vector, in absolute terms with the variable `USM_DRIFT_VECTOR`, or in relative terms with the variables `USM_DRIFT_VECTOR_ADD`.

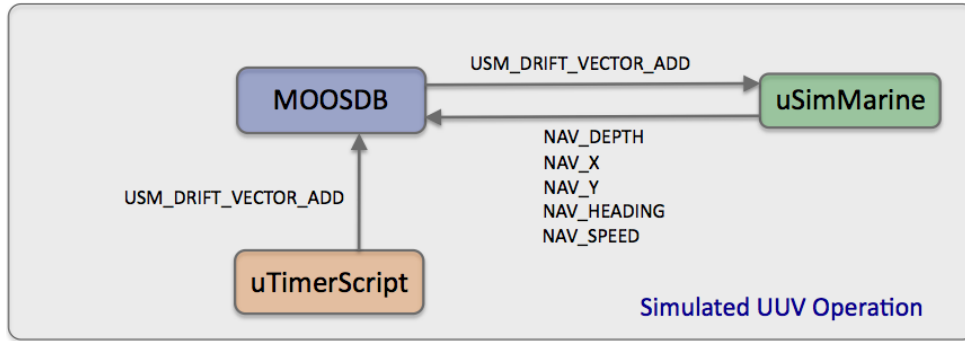


Figure 2: **Simulated Wind Gusts:** The **uTimerScript** application may be configured to post periodic sequences of external drift values, used by the **uSimMarine** application to simulate wind gust effects on its simulated vehicle.

The script in Listing 7 makes use of the **uSimMarine** interface by posting periodic drift vectors. It simulates a wind gust with a sequence of five posts to increase a drift vector (lines 18-22), and complementary sequence of five posts to decrease the drift vector (lines 24-28) for a net drift of zero at the end of each script execution.

Listing 9.7: A **uTimerScript** configuration for simulating simple wind gusts.

```

1  //-----
2  // uTimerScript configuration block
3
4  ProcessConfig = uTimerScript
5  {
6      AppTick    = 2
7      CommsTick  = 2
8
9      paused      = false
10     reset_max   = unlimited
11     reset_time  = end
12     delay_reset = 10:60
13     time_warp   = 0.25:2.0
14     script_name = WIND
15     script_atomic = true
16
17     randvar = varname=ANG, min=0, max=359, key=at_reset
18     randvar = varname=MAG, min=0.5, max=1.5, key=at_reset
19
20     event = var=DRIFT_VECTOR_ADD, val="$(ANG),{$(MAG)*0.2}", time=0
21     event = var=DRIFT_VECTOR_ADD, val="$(ANG),{$(MAG)*0.2}", time=2
22     event = var=DRIFT_VECTOR_ADD, val="$(ANG),{$(MAG)*0.2}", time=4
23     event = var=DRIFT_VECTOR_ADD, val="$(ANG),{$(MAG)*0.2}", time=6
24     event = var=DRIFT_VECTOR_ADD, val="$(ANG),{$(MAG)*0.2}", time=8
25
26     event = var=DRIFT_VECTOR_ADD, val="$(ANG),{$(MAG)*-0.2}", time=10
27     event = var=DRIFT_VECTOR_ADD, val="$(ANG),{$(MAG)*-0.2}", time=12
28     event = var=DRIFT_VECTOR_ADD, val="$(ANG),{$(MAG)*-0.2}", time=14
29     event = var=DRIFT_VECTOR_ADD, val="$(ANG),{$(MAG)*-0.2}", time=16
30     event = var=DRIFT_VECTOR_ADD, val="$(ANG),{$(MAG)*-0.2}", Time=18
31 }

```

The drift *angle* is chosen randomly in the range of [0,359] by use of the random variable macro `$(ANG)` defined on line 16. The peak *magnitude* of the drift vector is chosen randomly in the range of [0.5,1.5] with the random variable macro `$(MAG)` defined on line 17. Note that these two macros have their random values reset each time the script begins, by using the `key=at.reset` option, to ensure a stream of wind gusts of varying angles and magnitudes.

*The duration of each gust sequence also varies between each script execution. The default duration is about 20 seconds, given the timestamps of 0 to 18 seconds in lines 19-29. The `time_warp` option on line 12 affects the duration with a random value chosen from the interval of [0.25,2.0]. A time warp of 0.25 results in a gust sequence lasting about 80 seconds, and 2.0 results in a gust of about 10 seconds. The time between gust sequences is chosen randomly in the interval [10,60] by use of the `delay_restart` parameter on line 11. Used in conjunction with the `time_warp` parameter, the interval for possible observed delays between gusts is [5,240]. The `reset_time` parameter set to `end`, on line 10 is used to ensure that the script posts all drift vectors to avoid any accumulated drifts over time. The `reset_max` parameter is set to "unlimited" to ensure the script runs indefinitely.